CT-159 · DATA STRUCTURES

Trees

Lecture 9 — Binary Trees, Traversals & Binary Search Trees

Presenter: Dr. Muhammad Kamran

SECTION 01

Tree Concepts

A hierarchical structure — the first non-linear data structure we meet.

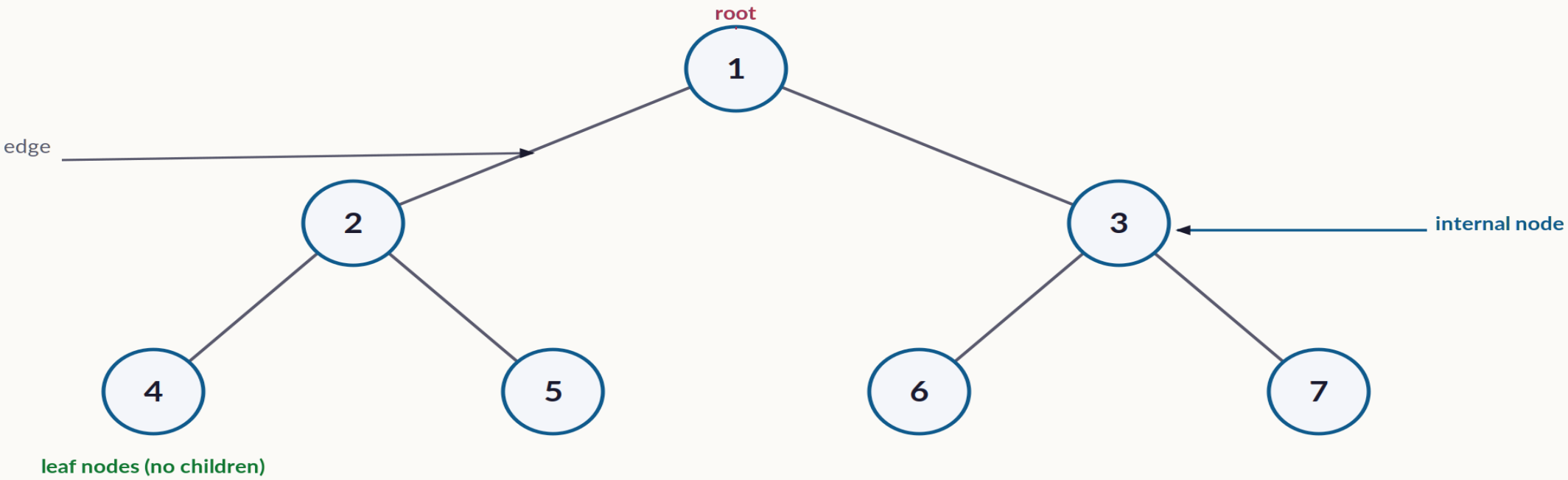
What is a Tree?

A tree is a hierarchy of nodes: one root at the top, with child nodes branching below.

- Unlike arrays, lists, stacks and queues, a tree is non-linear.
- Each node stores data and links to its children.
- There is exactly one path from the root to any node.

Anatomy of a Tree

Root, edges, internal nodes and leaves — the parts of every tree.



A tree is a hierarchy of nodes connected by edges, starting from a single root.

Each node holds data and links to its children. Nodes with no children are leaves.

In a binary tree, every node has at most two children (left and right).

Key Terms

Root	The topmost node; it has no parent.
Parent / Child	A node directly above / below another, connected by an edge.
Leaf	A node with no children.
Edge	The link connecting a parent to a child.
Height / Depth	Longest path to a leaf / distance from the root.
Subtree	Any node together with all of its descendants.

Binary Tree & Properties

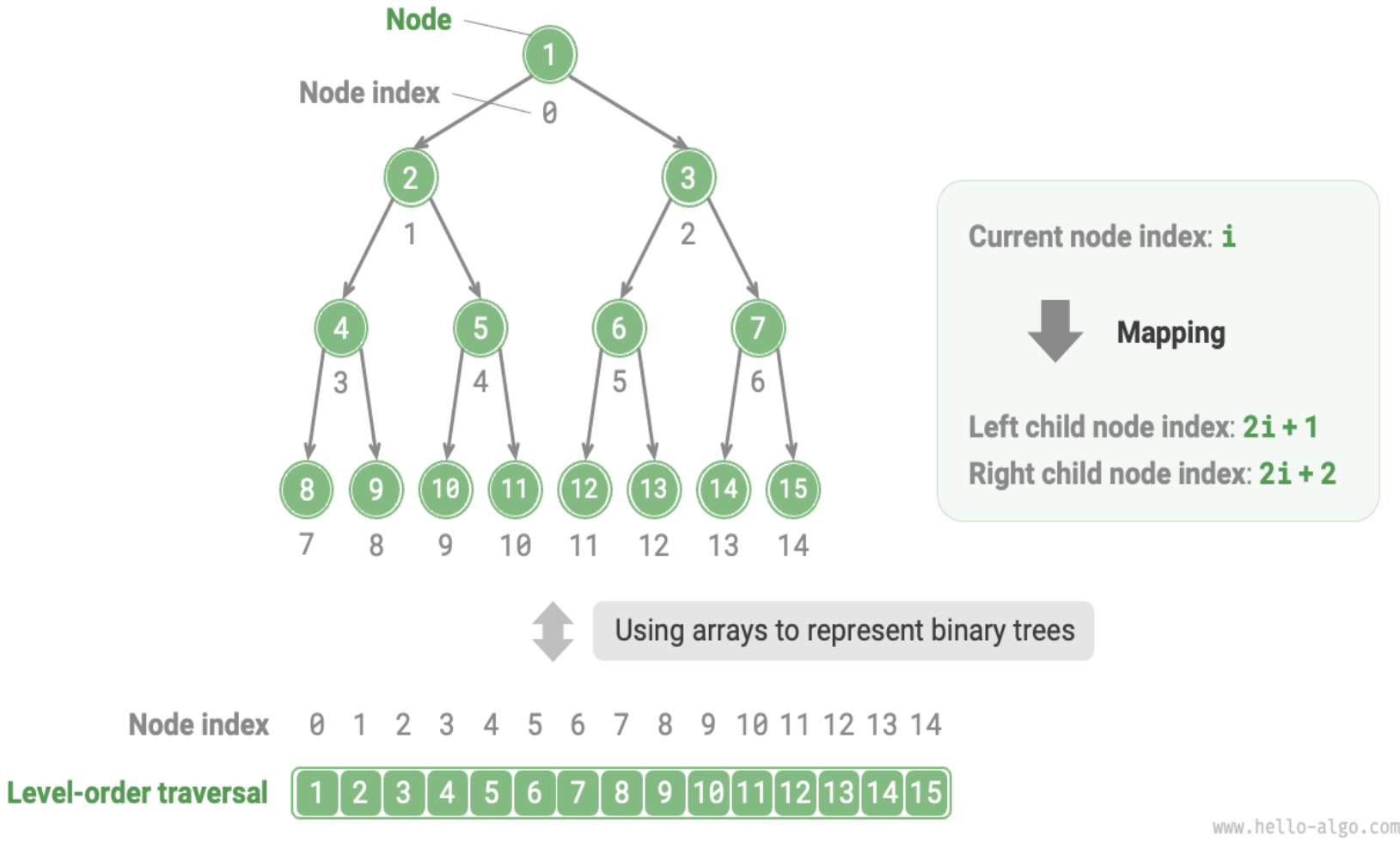
- A binary tree: every node has at most two children (left and right).
- Maximum nodes at level L = 2^L (root is level 0).
- Maximum nodes in a tree of height h = $2^{h+1} - 1$.
- A tree of n nodes has exactly n − 1 edges.



Each node stores its data plus two pointers: left child and right child.

A NULL pointer means there is no child on that side.

Array Representation



Types of Binary Tree

Full	Every node has either 0 or 2 children.
Complete	All levels filled except possibly the last, filled left to right.
Perfect	All internal nodes have 2 children and all leaves are on the same level.
Balanced	Left and right subtree heights differ by at most 1 (e.g. AVL).
Degenerate / Skewed	Each parent has one child — behaves like a linked list.

SECTION 02

Traversals

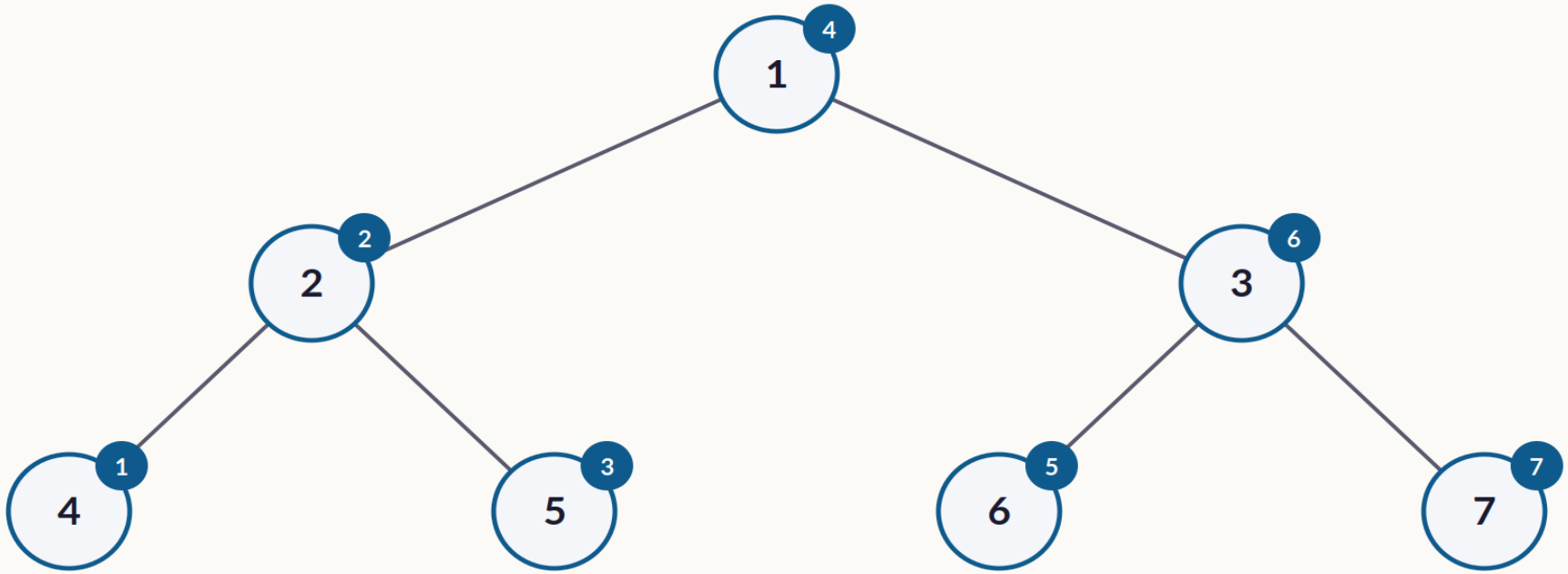
Four standard ways to visit every node in a tree.

Depth-First vs Breadth-First

- Depth-First (DFS): go as deep as possible before backtracking — in-order, pre-order, post-order.
- Breadth-First (BFS): visit level by level, top to bottom — also called level-order.
- DFS is naturally recursive (uses the call stack); BFS uses an explicit queue.

In-order Traversal

Left → Root → Right. On a BST this visits nodes in sorted order.



Rule: Left → Root → Right

Visit order: 4 2 5 1 6 3 7

In-order — Algorithm

Steps

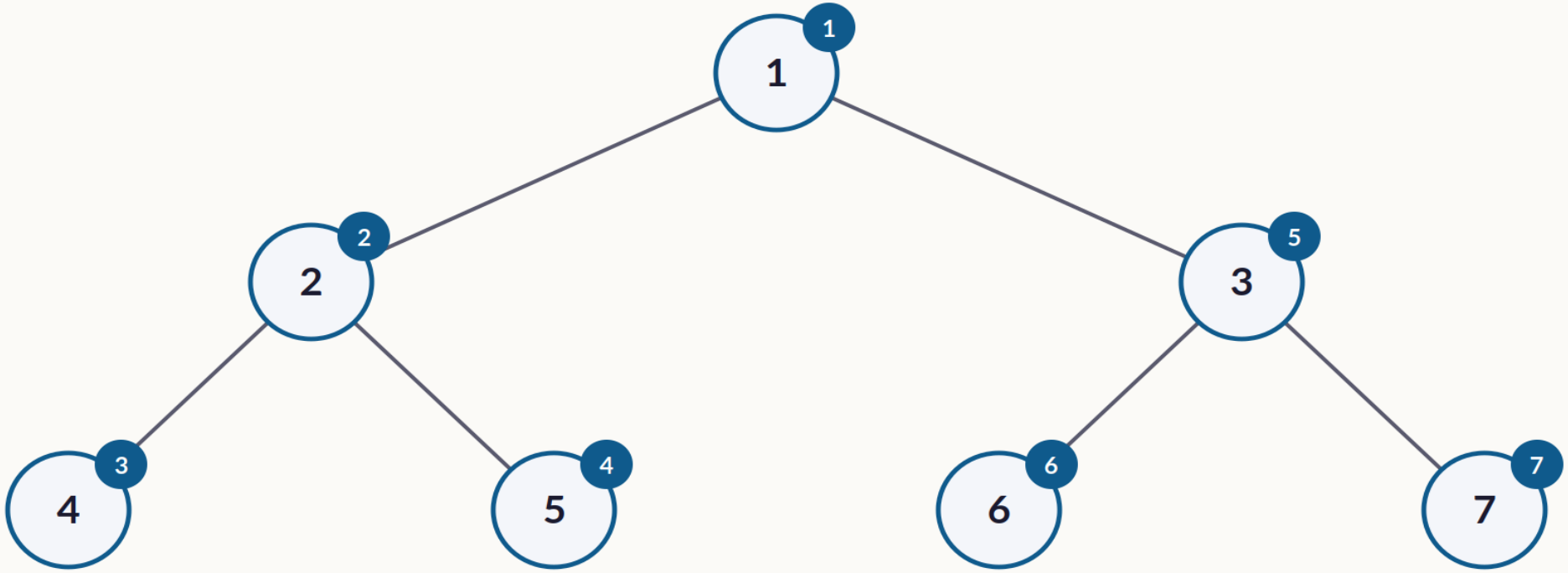
1. If node is NULL, return
2. Recurse in-order on the left child
3. Visit (print) the current node
4. Recurse in-order on the right child

```
// pseudocode
inorder(node):
    if node == NULL: return
    inorder(node.left)
    visit(node)
    inorder(node.right)
```

Time: O(n) — visits every node once. Space O(h) for the call stack.

Pre-order Traversal

Root → Left → Right. Useful for copying a tree or writing prefix expressions.



Rule: Root → Left → Right

Visit order: 1 2 4 5 3 6 7

Pre-order — Algorithm

Steps

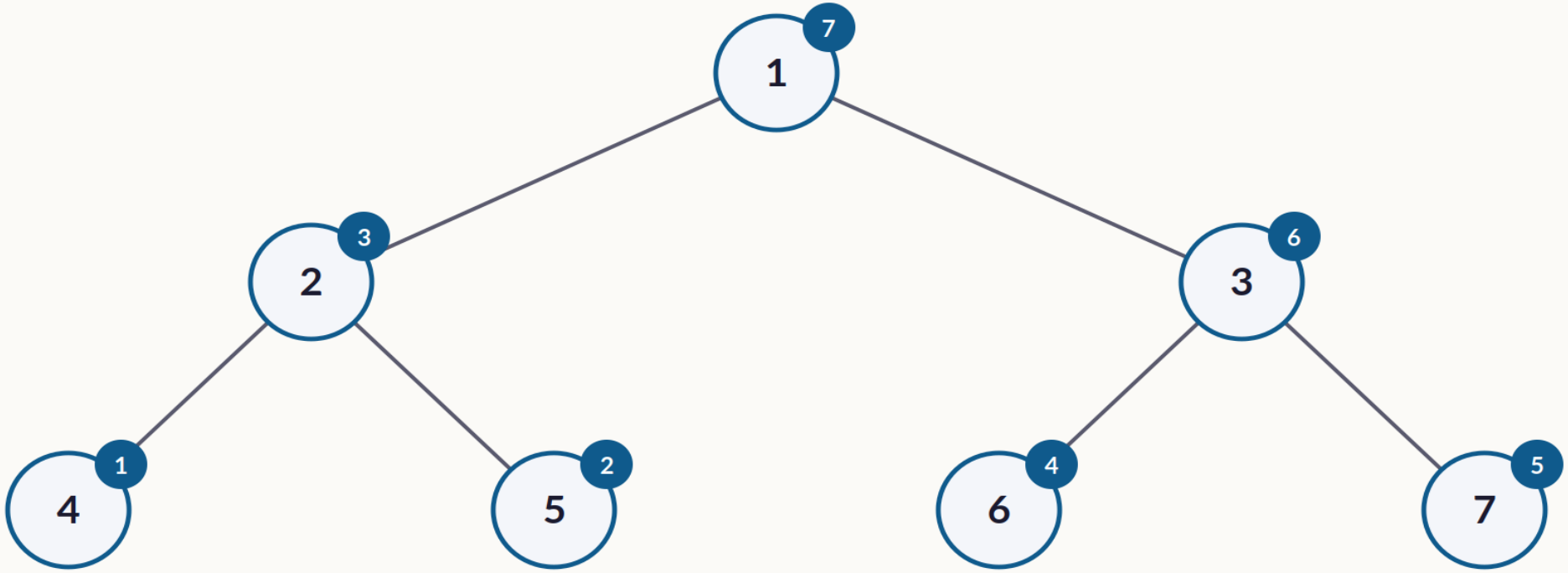
1. If node is NULL, return
2. Visit (print) the current node
3. Recurse pre-order on the left child
4. Recurse pre-order on the right child

```
// pseudocode
preorder(node):
    if node == NULL: return
    visit(node)
    preorder(node.left)
    preorder(node.right)
```

Time: O(n) time, O(h) space (h = tree height).

Post-order Traversal

Left → Right → Root. Useful for deleting a tree or evaluating expressions.



Rule: Left → Right → Root

Visit order: 4 5 2 6 7 3 1

Post-order — Algorithm

Steps

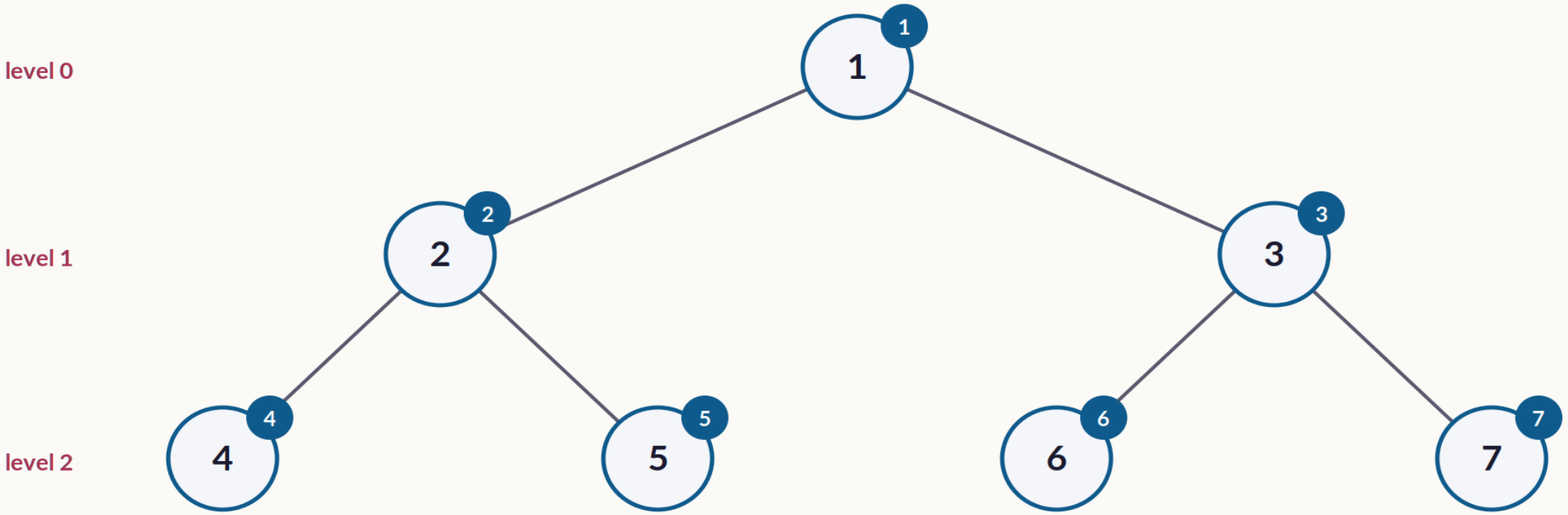
1. If node is NULL, return
2. Recurse post-order on the left child
3. Recurse post-order on the right child
4. Visit (print) the current node

```
// pseudocode
postorder(node):
    if node == NULL: return
    postorder(node.left)
    postorder(node.right)
    visit(node)
```

Time: O(n) time, O(h) space.

Level-order (BFS)

Visit level by level, left to right — using a queue.



Rule: visit level by level, left to right (uses a queue)

Visit order: 1 2 3 4 5 6 7

Level-order — Algorithm

Steps

1. Put the root into a queue
2. While the queue is not empty:
3. dequeue a node and visit it
4. enqueue its left child (if any)
5. enqueue its right child (if any)

```
// pseudocode

levelOrder(root):
    if root == NULL: return
    q = Queue(); q.enqueue(root)
    while not q.isEmpty():
        node = q.dequeue()
        visit(node)
        if node.left:  q.enqueue(node.left)
        if node.right: q.enqueue(node.right)
```

Time: O(n) time. Space O(n) — the queue can hold a whole level.

SECTION 03

Binary Search Trees

An ordered binary tree that makes search, insert and delete fast.

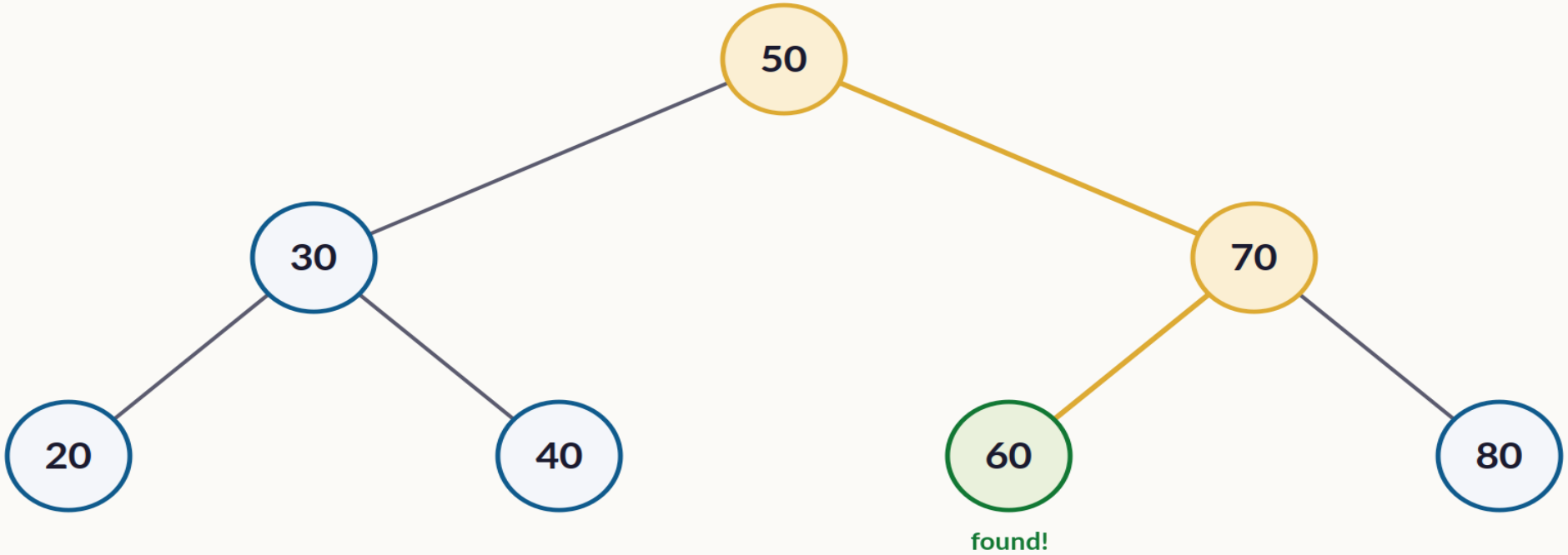
What is a Binary Search Tree?

A binary tree where, for every node, all left-subtree values are smaller and all right-subtree values are larger.

- This ordering lets us discard half the tree at each step, like binary search.
- Search, insertion and deletion all run in O(h) — the height of the tree.
- Balanced: h ≈ log n → O(log n). Skewed: h ≈ n → O(n) worst case.

BST — Search

Compare at each node and go left (smaller) or right (larger).



Search 60: compare at each node and go left (smaller) or right (larger).

60>50 → right; 60<70 → left; 60 = 60 → found. Only the height of the tree is visited.

Search — Algorithm

Steps

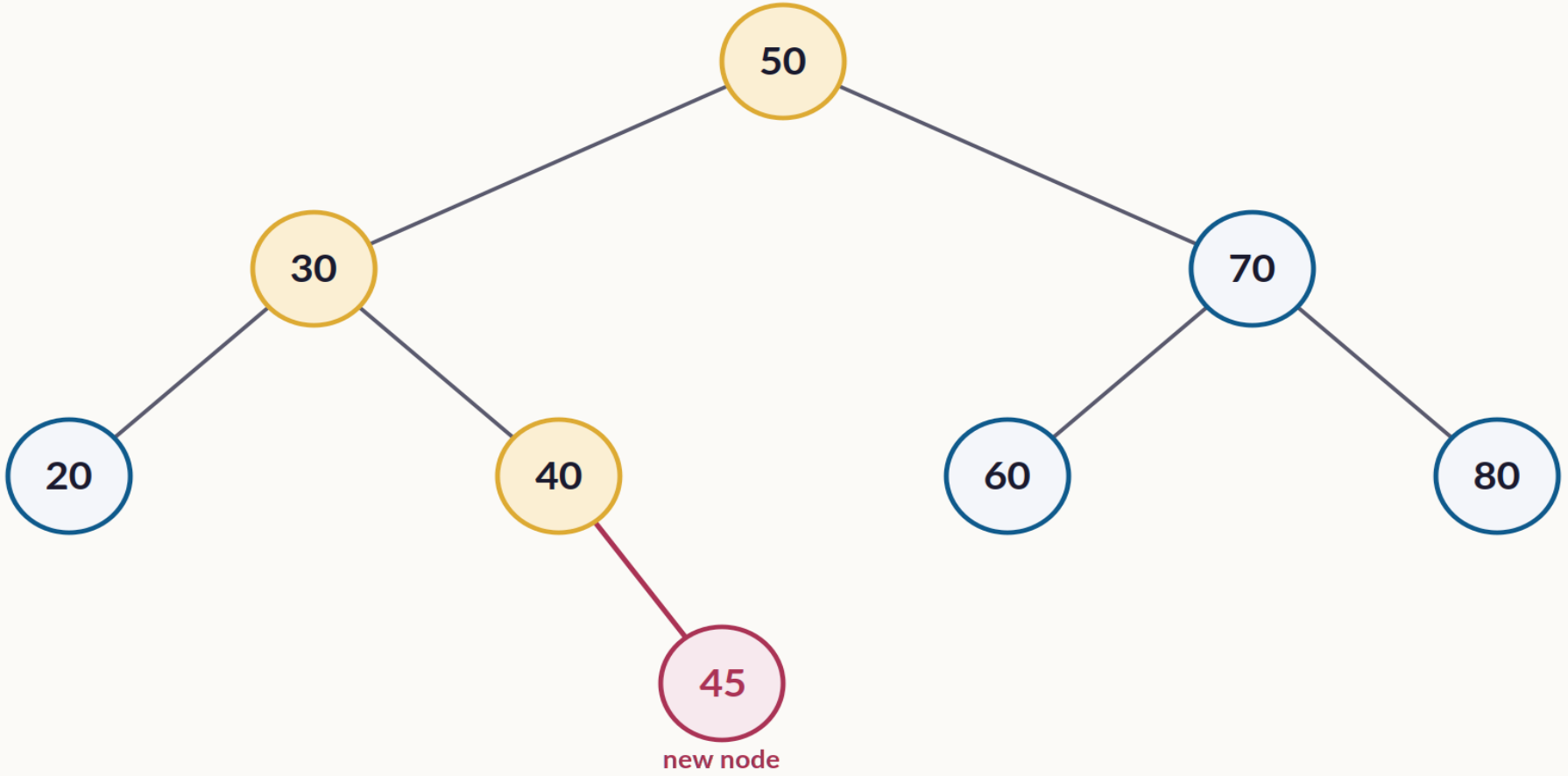
1. Start at the root
2. If key == node → found
3. If key < node → go left
4. If key > node → go right
5. Reached NULL → not found

```
// pseudocode
search(node, key):
    while node != NULL:
        if key == node.data: return node
        if key < node.data:
            node = node.left
        else:
            node = node.right
    return NULL
```

Time: O(h): O(log n) balanced, O(n) worst (skewed).

BST — Insertion

Search for the key's place, then attach the new node where the search ends.



Insert 45: 45>50? no→left; 45>30? yes→right; 45>40? yes→right (empty) → place here.

Insertion — Algorithm

Steps

1. If tree is empty, new node becomes root
2. Compare key with the current node
3. Smaller → go left; larger → go right
4. When you reach a NULL spot, insert there

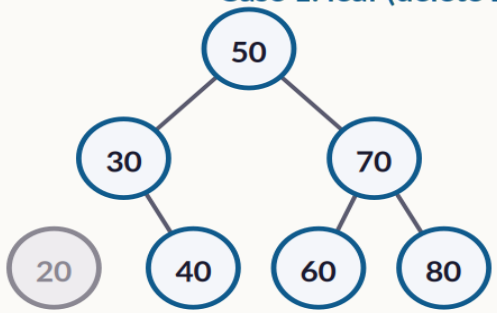
```
// pseudocode
insert(node, key):
    if node == NULL:
        return Node(key)
    if key < node.data:
        node.left = insert(node.left, key)
    else if key > node.data:
        node.right = insert(node.right, key)
    return node
```

Time: O(h): O(log n) balanced, O(n) worst.

BST — Deletion (3 cases)

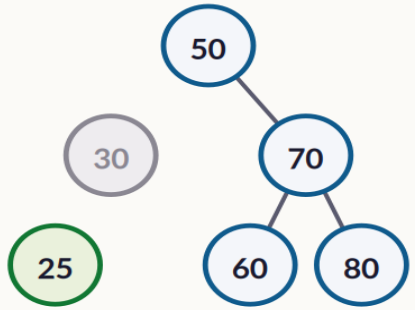
Leaf → remove; one child → child replaces it; two children → in-order successor.

Case 1: leaf (delete 20)



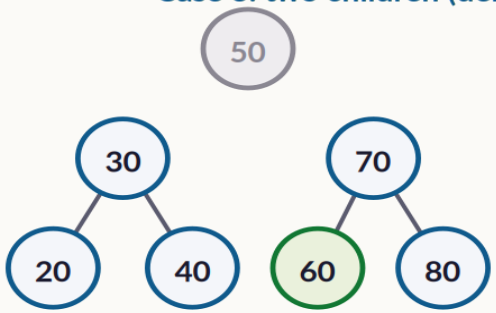
just remove it

Case 2: one child (delete 30)



child replaces it

Case 3: two children (delete 50)



successor takes place

Deletion has three cases:

1) Leaf → simply remove it. 2) One child → the child replaces the node.

3) Two children → replace with the in-order successor (smallest in the right subtree), then delete that.

Deletion — Algorithm

Steps

1. Find the node to delete
2. Leaf → just remove it
3. One child → link the child to the parent
4. Two children → copy in-order successor's value, then delete the successor

```
// pseudocode

delete(node, key):
    // ... find node ...
    if node has no child: remove node
    else if node has one child:
        replace node with that child
    else:                          // two
children
        succ = min(node.right)  // in-order
successor
        node.data = succ.data
        node.right = delete(node.right,
succ.data)
```

Time: O(h): O(log n) balanced, O(n) worst.

BST Operations — Complexity

Search	O(h) → O(log n) balanced, O(n) skewed
Insertion	O(h) → O(log n) balanced, O(n) skewed
Deletion	O(h) → O(log n) balanced, O(n) skewed
Traversal (any)	O(n) — every node visited once

This is why balancing matters: AVL and Red-Black trees keep h ≈ log n so operations stay fast.

Where Trees Are Used

- File systems — folders and files form a tree.
- Databases and search — B-trees, binary search trees.
- Expression parsing and compilers — syntax trees.
- Priority queues and scheduling — heaps.
- Routing, AI decision trees, auto-complete (tries).

SECTION 04

Practice It Yourself

Tree problems are recursion problems — draw the tree and trust the recursion.

Problems to Try

Sketch the tree and trace the recursion by hand before you code.

LeetCode 226	Invert Binary Tree	Easy	Swap every node's children (mirror the tree).
LeetCode 104	Maximum Depth	Easy	Find the height of a binary tree.
LeetCode 94	Inorder Traversal	Easy	Return the in-order visit sequence.
LeetCode 700	Search in a BST	Easy	Classic BST search.
LeetCode 98	Validate BST	Medium	Check the BST ordering property holds.

LeetCode 226 — The Approach

Problem (in short): invert a binary tree — mirror it so every left child becomes a right child and vice versa.

- At each node, swap its left and right child pointers.
- Then recurse: invert the left subtree and the right subtree the same way.
- The base case is a NULL node — nothing to do, just return.
- Every node is visited once → O(n) time, O(h) recursion depth.

YOUR TURN

On paper, invert the tree [4,2,7,1,3,6,9] by hand. What's the new root's left and right child?

LeetCode 226 — The Code

```
TreeNode* invertTree(TreeNode* root) {
    if (root == nullptr)           // base case
        return nullptr;

    swap(root->left, root->right);   // swap children

    invertTree(root->left);          // recurse left
    invertTree(root->right);         // recurse right

    return root;
}
```

Why It Works & How Fast

The swap	Swapping the two child pointers mirrors that node's two subtrees.
Recursion	Doing the same at every node mirrors the whole tree, top to bottom.
Time	O(n) — each node is visited exactly once.
Space	O(h) — the recursion stack, where h is the tree's height.

Where to Get Help

Watch the idea only after you've tried it — then close the video and code it yourself.

NeetCode	Invert Binary Tree, clearly explained. Search: "NeetCode Invert Binary Tree"
Abdul Bari	Trees, BST & traversals. Search: "Abdul Bari Binary Search Tree"
William Fiset	Full tree & BST playlist. Search: "William Fiset Binary Tree"

YOUR TURN

Course rule: draw the tree and trace each problem for 20 minutes before watching anything — and never paste it into an AI.

LECTURE 9 — RECAP

What we covered

- Tree: non-linear hierarchy of nodes & edges
- Binary tree: at most two children per node
- Traversals: in / pre / post-order (DFS) + level-order (BFS)
- BST keeps left < node < right
- Search / insert / delete are O(h)
- Balanced h ≈ log n; skewed h ≈ n

Next: BSTs